

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/216301461>

Feature Construction and Selection Using Genetic Programming and a Genetic Algorithm

Conference Paper in Lecture Notes in Computer Science · April 2003

DOI: 10.1007/3-540-36599-0_21

CITATIONS

59

READS

870

2 authors, including:



[Matthew G. Smith](#)

University of the West of England, Bristol

6 PUBLICATIONS 235 CITATIONS

SEE PROFILE

Feature Construction and Selection using Genetic Programming and a Genetic Algorithm

Matthew G. Smith and Larry Bull

Faculty of Computing, Engineering & Mathematical Sciences,
University of the West of England, Bristol BS16 1QY, U.K.

Matt-Smith@bigfoot.com, Larry.Bull@uwe.ac.uk

Abstract. The use of machine learning techniques to automatically analyse data for information is becoming increasingly widespread. In this paper we examine the use of Genetic Programming and a Genetic Algorithm to pre-process data before it is classified using the C4.5 decision tree learning algorithm. The Genetic Programming is used to construct new features from those available in the data, a potentially significant process for data mining since it gives consideration to hidden relationships between features. The Genetic Algorithm is used to determine which such features are the most predictive. Using ten well-known datasets we show that our approach, in comparison to C4.5 alone, provides marked improvement in a number of cases.

1. Introduction

Classification is one of the major tasks in data mining, involving the prediction of class value based on information about some other attributes. The process is a form of inductive learning whereby a set of pre-classified training examples are presented to an algorithm which must then generalise from those seen to correctly categorise unseen examples. One of the most commonly used forms of classification technique is the decision tree learning algorithm C4.5 [7]. In this paper we examine the use of Genetic Programming (GP) [5] and a Genetic Algorithm (GA)[3] to improve the performance of C4.5 through feature *construction* and feature *selection*. Feature construction is a process that aims to discover hidden relationships between features, inferring new composite features. In contrast, feature selection is a process that aims to refine the list of features used thereby removing potential sources of noise and ambiguity. We use GP individuals consisting of a number of separate trees/automatically defined functions (ADFs) [5] to construct features for C4.5. A GA is then used to select over the original and constructed features for a final hybrid C4.5 classifier system. Results show that the system is able to outperform standard C4.5 on a number of datasets held at the UCI repository (<http://www.ics.uci.edu/~mllearn/MLRepository.html>).

Raymer et al. [8] have used ADFs for feature *extraction* in conjunction with the k-nearest-neighbour algorithm. Feature extraction replaces an original feature with the result from passing it through a functional mapping. In Raymer et al.'s approach each feature is altered by an ADF, evolved for that feature only, with the aim of increasing the separation of pattern classes in the feature space; for problems with n features, individuals consist of n ADFs. Ahluwalia and Bull [1] extended Raymer et al.'s approach by coevolving the ADFs for each feature and adding an extra coevolving GA population of feature selectors; extraction and selection occurred simultaneously in $n+1$ populations. For other (early) examples of evolutionary computation approaches to data mining see [9] for a GA-based feature selection approach using k-nearest-neighbour and [4] for a similar GA-based approach also using k-nearest-neighbour. Since undertaking the work presented here we have become aware of Vafaie and DeJong's [10] combination of GP and a GA for use with C4.5. They used the GA to perform feature selection for a face recognition dataset where feature subsets were evaluated through their use by C4.5. GP individuals were then evolved which contained a variable number of ADFs to construct new features from the selected subset, again using C4.5. Our approach is very similar to Vafaie and DeJong's but the feature operations are reversed such that feature construction occurs before selection. We find that our approach performs as well or better than Vafaie and DeJong's.

The paper is arranged as follows: the next section describes the approach; section 3 presents results from its use on a number of well-known datasets; and finally, all findings are discussed.

2. The GAP Algorithm

In this work we have used the WEKA [see 11] implementation of C4.5, known as J48, to examine the performance of our Genetic Algorithm and Programming (GAP) approach. The approach consists of two phases:

2.1 Feature Creation

A population of 101 genotypes is created at random. Each genotype consists of n trees, where n is the number of numeric valued features in the dataset, subject to a minimum of 7. This minimum is chosen to ensure that the initial population contains a significant number of compound features. A tree can be either an original feature or an ADF. That is, a genotype consists of n GP trees, each of which may contain 1 or more nodes. The chance of a node being a leaf node (a primitive attribute) is determined by:

$$P_{\text{leaf}} = 1 - \frac{1}{(\text{depth} + 1)}$$

Where *depth* is the depth of the tree at the current node. Hence a root node will have a depth of 1, and therefore a probability of 0.5 of being a leaf node. Nodes at

depth 2 will have a 0.67 probability of being a leaf node, and so on. If a node is a leaf node, it takes the value of one of the original features chosen at random. Otherwise, a function is randomly chosen from the set $\{*, /, +, -, \%\}$ and two child nodes are generated. In this manner there is no absolute limit placed on the depth any one tree may reach but the average depth is limited. During the initial creation no two trees in a single genotype are allowed to be alike, though this restriction is not enforced in later stages. Additionally, nodes with '-', '%', or '/' for functions cannot have child nodes that are equal to each other. In order to enforce this child nodes within a function '*' or '+' are ordered alphabetically to enable comparison (e.g. $[\text{width} + \text{length}]$ will become $[\text{length} + \text{width}]$).

An individual is evaluated by constructing a new dataset with one feature for each tree in the genotype. This dataset is then passed to a C4.5(J48) classifier (using default parameters), whose performance on the dataset is evaluated using 10-fold cross validation. The percentage correct is then assigned to the individual and used as the fitness score.

Once the initial population has been evaluated, several generations of selection, crossover, mutation and evaluation are performed. After each evaluation, if the fittest individual in the current generation is fitter than the fittest so far, a copy of it is set aside and the generation noted. The evolutionary process continues until the following conditions are met: at least 10 generations have passed, and the fittest individual so far is at least 6 generations old. There is no maximum generation, but in practice very rarely have more than 50 generations been necessary, and often fewer than 30 are required.

We use tournament selection here, with tournament size 8 and a 0.3 probability of the fittest individual winning (otherwise a 'winner' is selected at random from the tournament group). There is a 0.6 probability of two-point crossover occurring between the ADFs of the two selected parents (whole trees are exchanged between genotypes) and a 0.6 probability that crossover will occur between two ADFs at a randomly chosen locus (sub-trees are exchanged between trees). Mutation is used with probability 0.008 per node where a randomly created subtree replaces the subtree under the selected node. We also use a form on inversion with probability 0.2 whereby the order of the ADFs between two randomly chosen loci is reversed. Experimentation on varying these parameters has found the algorithm to be fairly robust to their setting (not shown).

Once the termination criteria have been met the fittest individual is used to seed the feature selection stage.

2.2 Feature Selection

The fittest individual from the feature creation stage (ties broken randomly) is analysed to see if any of the original features do not appear to be used. If there are any missing, sufficient trees are added to ensure that every original feature in the dataset appears at least once (the new trees are not randomly generated as in the feature creation stage, but have single nodes containing the required attribute). This extended genotype (up to twice as long as the fittest individual of the feature creation stage) replaces the initial individual and is used as the basis of the second stage.

A new dataset is constructed with one attribute for every tree in the extended genotype. In an attempt to reduce over-fitting of the data, the order of the dataset is randomised at this point. This has the effect of providing a different split of the data for 10-fold cross validation during the selection stage, giving the algorithm a chance of recognising trees that performed well only due to the particular data partition in the creation stage. As a result of the randomisation, it is usually the case that the fitness score of individuals in the selection stage is less than that of individuals in the creation stage but solutions should be more robust.

For the GA a population of 101 bit strings is randomly created. The strings have the same number of bits as the genotype has trees – there is one bit for every attribute (some composite, some primitive attributes). The last member of the population, the 101st bit string, is not randomly created but is initialised to all ones. This ensures that there are no missing alleles at the start of the selection.

Once the entire population has been created, each genotype is evaluated and assigned a fitness score that is used in selecting the parents of the next generation. A GA bit string is evaluated by taking a copy of the parent dataset and removing every attribute that has a '0' in the corresponding position in the bit string. As in the feature creation stage, this dataset is then passed to a C4.5(J48) classifier whose performance on the dataset is evaluated using 10-fold cross validation. The percentage correct is then assigned to the bit string and used as the fitness score.

If the fittest individual in the current generation has a higher fitness score than the fittest so far (from the selection stage, ignoring genotypes from the feature creation stage), or it has the same fitness score and fewer '1's, a copy of it is set aside and the generation noted. As in the feature creation stage the cycles of selection, crossover, mutation, and evaluation continue until the following conditions are met: at least 10 generations have passed, and the fittest individual so far is at least 6 generations old.

The selection scheme is the same as for the creation stage. There is a 0.6 probability of two-point crossover and a 0.005 per bit probability of mutation.

3. Experimentation

We have used ten well-known data sets from the UCI repository to examine the performance of the GAP algorithm. The UCI datasets were chosen because they consisted entirely of numeric attributes (though the algorithm can handle some nominal attributes, as long as there are two or more numeric attributes present) and had no missing values (missing values could be handled with only a minor modification to the code). Table 1 shows the details of the ten datasets used here.

Table 1: UCI dataset information.

Dataset	# Numeric features	# Nominal features	# Classes	# Instances
BUPA Liver Disorder	6	0	2	345
Glass Identification	9	0	6	214
Ionosphere	34	0	2	351
New Thyroid	5	0	3	215
Pima Indians Diabetes	8	0	2	768
Sonar	60	0	2	208
Vehicle	18	0	4	846
Wine Recognition	13	0	3	178
Wisconsin Breast Cancer (New)	30	0	2	569
Wisconsin Breast Cancer (Original)	9	0	2	699

For performance comparisons the tests were performed using ten-fold cross-validation. An additional set of ten runs using ten-fold cross validation were made (a total of twenty runs, two sets of ten-fold cross-validation) to allow a paired *t*-test to establish significant improvement over C4.5(J48).

3.1 Results

The highest classification score for each dataset is shown in Table 2 in bold underline. The first two columns show the performance of the GAP algorithm on the training data and the last column shows the results of the paired *t*-test. Results that are significant at the 95% confidence level are shown in bold.

The GAP algorithm out-performs C4.5(J48) on eight out of ten datasets, and provides a significant improvement on three (Glass Identification, New Thyroid, and Wisconsin Breast Cancer Original) – two of which are significant at the 99% confidence level. There are no datasets on which the GAP algorithm performs significantly worse than C4.5(J48) alone.

The standard deviation of the GAP algorithm’s results do not seem to differ greatly from that of C4.5(J48); there are five datasets where the GAP algorithms’ standard deviation is greater and five where it is smaller. This is perhaps the most surprising aspect of the results, given that the GAP algorithm is unlikely to produce the same result twice when presented with exactly the same data, whereas C4.5(J48) will always give the same result if presented with the same data.

As noted in the introduction, Vafaie and DeJong [10] have used a very similar approach to improve the performance of C4.5. They use feature selection (GA) followed by feature construction (GP). We have examined the performance of our algorithm as described above with the two processes occurring in the opposite order. Results indicate that GAP gives either equivalent (e.g. Wisconsin Breast Cancer) or better performance (e.g. New Thyroid) (Table 3). We suggest this is due to GAP’s potential to construct new features in a less restricted way, i.e. its ability to use all the original features during the create stage. For instance, on the New Thyroid dataset the

select stage will always remove either feature 1 or feature 2 thus preventing the create stage from being able to construct the apparently useful feature “2”/”1” (see section 3.2). That is, on a number of datasets there is a significant difference (at the 95% confidence level) in the results brought about by changing the order of the stages.

Table 2: Comparative performance of GAP algorithm and C4.5 (J48).

Dataset	GAP Train	S.D.	GAP Test	S.D.	C4.5 (J48)	S.D.	Paired <i>t</i> -test
BUPA Liver Disorder	75.04	2.37	65.97	11.27	<u>66.37</u>	8.86	-0.22
Glass Identification	78.17	1.95	<u>73.74</u>	9.86	68.28	8.86	3.39
Ionosphere	95.99	0.87	89.38	4.76	<u>89.82</u>	4.79	-0.34
New Thyroid	98.22	0.68	<u>96.27</u>	4.17	92.31	4.14	3.02
Pima Indians Diabetes	78.15	0.96	<u>73.50</u>	4.23	73.32	5.25	0.19
Sonar	92.33	1.27	<u>73.98</u>	11.29	73.86	10.92	0.05
Vehicle	78.82	1.21	<u>72.46</u>	4.72	72.22	3.33	0.20
Wine Recognition	98.47	0.80	<u>94.68</u>	5.66	93.27	5.70	0.85
Wisconsin Breast Cancer (New)	97.86	0.47	<u>95.62</u>	2.89	93.88	4.22	1.87
Wisconsin Breast Cancer (Original)	97.65	0.38	<u>95.63</u>	1.58	94.42	3.05	2.11

Table 3: Comparison of ordering of Create and Select stages

Dataset	Create then Select		Select then Create	
	Test	S.D.	Test	S.D.
BUPA Liver Disorder	65.97	11.27	<u>67.42</u>	8.23
Glass Identification	<u>73.74</u>	9.86	68.75	6.36
Ionosphere	<u>89.38</u>	4.76	89.02	4.62
New Thyroid	<u>96.27</u>	4.17	93.67	5.82
Pima Indians Diabetes	73.50	4.23	<u>74.09</u>	4.46
Sonar	<u>73.98</u>	11.29	73.16	9.05
Vehicle	72.46	4.72	72.46	3.01
Wine Recognition	94.68	5.66	<u>94.69</u>	3.80
Wisconsin Breast Cancer (New)	95.62	2.89	<u>95.88</u>	3.15
Wisconsin Breast Cancer (Original)	<u>95.63</u>	1.58	95.13	2.16

3.2 Analysis

We were interested in whether the improvement over C4.5(J48) is simply the result of the selection stage choosing an improved subset of features and discarding the new constructed features. An analysis of the attributes output by the GAP classifier algorithm, and the use made of them in C4.5’s decision trees, shows this is not the case.

As noted above, the results in Table 2 were obtained from twenty runs on each of ten UCI datasets, i.e. a total of two hundred individual solutions. In those two hundred individuals there are a total of 2,425 trees: 982 ADFs and 1,443 original features - a ratio of roughly two constructed features to three original features. *All but two of the two hundred individuals contained at least one constructed feature.* Table 4 gives details of the average number of ADFs per individual for each dataset (the number of original features used is not shown).

Table 4: Analysis of the constructed features for each data set.

Dataset	Results			
Name	# Features in Dataset	Average # Features	Average # ADFs	Minimum # ADFs
BUPA Liver Disorder	6	4.9	2.6	1
Glass Identification	9	7.1	3.1	1
Ionosphere	34	19.7	7.6	4
New Thyroid	5	3.2	2.3	1
Pima Indians Diabetes	8	6.4	2.7	1
Sonar	60	38.0	13.6	5
Vehicle	18	16.3	5.5	2
Wine Recognition	13	5.2	2.1	0
Wisconsin Breast Cancer (New)	30	15.7	7.0	4
Wisconsin Breast Cancer (Original)	9	5.0	2.9	1

Knowing that the feature selection stage continues as long as there is a reduction in the number of attributes without reducing the fitness score, we can assume that C4.5(J48) is making good use of all the attributes in most if not all of the winning individuals. This can be demonstrated by looking in detail at the attributes in a single winner, and the decision tree created by C4.5(J48).

One of the best performers on the New Thyroid dataset had three trees, two of them ADFs and hence constructed features. The original dataset contains five features and the class:

1. T3-resin uptake test. (A percentage)
2. Total Serum thyroxin as measured by the isotopic displacement method.
3. Total serum triiodothyronine as measured by radioimmuno assay.
4. Basal thyroid-stimulating hormone (TSH) as measured by radioimmuno assay.
5. Maximal absolute difference of TSH value after injection of 200 micro grams of thyrotropin-releasing hormone as compared to the basal value.

Class attribute. (1 = normal, 2 = hyper, 3 = hypo)

In the chosen example the newly constructed features were:

- “5” becomes Feat0
- (“1”/’4”)*’2”) becomes Feat1
- (“2”/’1”) becomes Feat2

The decision tree created by C4.5 (the numbers after the class prediction indicate the count of train instances correctly / incorrectly classified) was:

```
Feat1 <= 222.352941: hypo (19.0)
Feat1 > 222.352941
|   Feat2 <= 0.113761: normal (102.0/1.0)
|   Feat2 > 0.113761
|   |   Feat0 <= 1.1: hyper (26.0)
|   |   Feat0 > 1.1: normal (3.0)
```

It is apparent that C4.5(J48) is using the constructed features to classify a large majority of the instances, and only referring to one of the original features (Feat0, or the fifth feature in the dataset) in 29 of 150 cases.

3.3 A Rough Comparison to other Algorithms

Table 5 presents a number of published results we have found regarding the same ten UCI datasets using other machine learning algorithms. It should be noted that in the table are the results for C4.5 that were not obtained using J48. Cells in the table are left blank where algorithms were not tested on the dataset in question. The highest classification score for each dataset is shown in bold underline.

Table 5: Performance of GAP and other algorithms on the UCI datasets.

Dataset	GAP	C4.5 (J48)	C4.5	HIDER	XCS	O. F. ARTMAP	LVSM
BUPA Liver Disorder	65.97	66.37	65.27	64.29	<u>67.85</u>	57.01	68.68
Glass Identification	<u>73.74</u>	68.28	67.27	70.59	72.53	69.56	
Ionosphere	89.38	<u>89.82</u>					87.75
New Thyroid	<u>96.27</u>	92.31					
Pima Indians Diabetes	73.50	73.32	67.94	74.1	68.62	69.8	<u>78.12</u>
Sonar	73.98	73.86	69.69	56.93	53.41	<u>79.96</u>	
Vehicle	<u>72.46</u>	72.22					
Wine Recognition	94.68	93.27	93.29	96.05	92.74	<u>98.27</u>	
Wisconsin Breast Cancer (New)	<u>95.62</u>	93.88					
Wisconsin Breast Cancer (Original)	95.63	94.42	93.72	95.71	<u>96.27</u>	94.39	

The results for C4.5, HIDER and XCS were obtained from [2], those for O.F. ARTMAP ('Ordered Fuzzy ARTMAP', a neural network algorithm) from [1] and LVSM (Lagrangian Support Vector Machines) from [6].

The differences between the reported results for C4.5 and those for C4.5 as used in this paper (J48, the WEKA implementation of C4.5) are likely to arise from different data partitions used for the tests (the most notable being the 5.38% difference in results for Pima Indians Diabetes). This discrepancy highlights the dangers inherent in comparing results with published data – the comparison should be seen as purely informal. The only comparisons that can be relied upon are those between the GAP classifier and C4.5(J48) as these have been performed using exactly the same procedure and data partitions. The results are by no means an exhaustive list of current machine learning algorithms, nor are they guaranteed to be the best performing algorithms available, but they give some indication of the relative performance of our approach – which appears to be very good.

4. Conclusion

In this paper we have presented an approach to improve the classification performance of the well-known induction algorithm C4.5. We have shown that GP individuals consisting of multiple trees/ADFs can be used for effective feature creation and that solutions, combined with feature selection via a GA, can give significant improvements to the classification accuracy of C4.5.

Future work will apply our approach to other datasets and data mining algorithms.

References

1. Ahluwalia, M. & Bull, L. (1999) Co-Evolving Functions in Genetic Programming: Classification using k-nearest neighbour. In W. Banzhaf, J. Daida, G. Eiben, M-H. Garzon, J. Honavar, K. Jakeila, R. Smith (eds) *GECCO-99: Proceedings of the Genetic and Evolutionary Computation Conference*. Morgan Kaufmann, pp. 947–952.
2. Dixon, P. W., Corne, D. W., & Oates, M. J. (2001) A Preliminary Investigation of Modified XCS as a Generic Data Mining Tool. In P-L. Lanzi, W. Stolzmann, S. Wilson (eds) *Advances in Learning Classifier Systems*. Springer, pp.133-151.
3. Holland, J.H. (1975) *Adaptation in Natural and Artificial Systems*. Univ. Michigan.
4. Kelly, J.D. & Davis, L. (1991) Hybridizing the Genetic Algorithm and the K Nearest Neighbors Classification Algorithm. In R. Belew & L. Booker (eds) *Proceedings of the Fourth International Conference on Genetic Algorithms*. Morgan Kaufmann, pp377-383.
5. Koza, J.R. (1992) *Genetic Programming*. MIT Press.
6. Mangasarian, O. L. & Musicant, D. R. (2001) Lagrangian support vector machines. *Journal of Machine Learning Research* 1:161-177.
7. Quinlan, J.R. (1993) *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
8. Raymer, M.L., Punch, W., Goodman, E.D. & Kuhn, L. (1996) Genetic Programming for Improved Data Mining - Application to the Biochemistry of Protein Interactions. In J.R. Koza, K. Deb, M. Dorigo, D.B. Fogel, M.Garzon, H. Iba & R. Riolo (eds) *Proceedings of the Second Annual Conference on Genetic Programming*, Morgan Kaufmann, pp375-380.

9. Siedlecki, W. & Sklansky, J. (1988) On Automatic Feature Selection. *International Journal of Pattern Recognition and Artificial Intelligence* 2:197-220.
10. Vafaie, H. & De Jong, K. 1995. Genetic Algorithms as a Tool for Restructuring Feature Space Representations. In *Proceedings of the International Conference on Tools with A.I.* IEEE Computer Society Press.
11. Witten, I.H. & Frank, E. (2000) *Data Mining: Practical Machine Learning Tools and Techniques with Java Implementations*. Morgan Kaufmann.